Namespace SixLabors.PolygonClipper

Classes

Contour
: Represents a single polygon ring (outer contour or hole).

Polygon
: Represents a complex polygon.

PolygonClipper
: Performs boolean operations on polygons.

PolygonStroker
: Generates polygonal stroke geometry for contours with configurable joins and caps.

StrokeOptions
: Provides configuration options for geometric stroke generation.

Structs

Box2
: Represents a bounding box.

Vertex
: Represents a two-dimensional vertex with X and Y coordinates.

Enums

BooleanOperation
: Specifies the type of boolean operation to perform on polygons.

LineCap
: Specifies the shape to be used at the ends of open lines or paths when stroking.

LineJoin
: Specifies how the connection between two consecutive line segments (a join) is rendered when stroking paths or polygons.

ResultTransition
: Represents the result transition for a sweep event.

Enum BooleanOperation

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Specifies the type of boolean operation to perform on polygons.

```
public enum BooleanOperation
```

Fields

`Difference = 2`

The difference operation, which subtracts the clipping polygon from the subject polygon.

`Intersection = 0`

The intersection operation, which results in the area common to both polygons.

`Union = 1`

The union operation, which results in the combined area of both polygons.

`Xor = 3`

The exclusive OR (XOR) operation, which results in the area covered by exactly one polygon, excluding the overlapping areas.

Struct Box2

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Represents a bounding box.

```
public readonly struct Box2 : IEquatable<Box2>
```

Implements
IEquatable<Box2>

Inherited Members
ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Box2(in Vertex)

Initializes a new instance of the Box2 struct.

```
public Box2(in Vertex vector)
```

Parameters

`vector` Vertex

The xy-coordinate.

Box2(in Vertex, in Vertex)

Initializes a new instance of the Box2 struct.

```
public Box2(in Vertex min, in Vertex max)
```

Parameters

`min` Vertex

The minimum xy-coordinate.

`max` Vertex

The maximum xy-coordinate.

Fields

Max

Gets the maximum xy-coordinate.

```
public readonly Vertex Max
```

Field Value

Vertex

Min

Gets the minimum xy-coordinate.

```
public readonly Vertex Min
```

Field Value

Vertex

Properties

Invalid

Gets an invalid bounds instance.

```
public static Box2 Invalid { get; }
```

Property Value

Box2

Methods

Add(in Box2)

Adds another bounding box to this instance.

```
public Box2 Add(in Box2 other)
```

Parameters

`other` Box2

The other box.

Returns

Box2

The summed Box2.

Contains(in Box2)

Returns true if the box contains another box.

```
public bool Contains(in Box2 bounds)
```

Parameters

`bounds` Box2

The other box.

Returns

bool

true if the box contains the other box; otherwise, false.

Contains(in Vertex)

Returns true if the point lies within the box.

```
public bool Contains(in Vertex point)
```

Parameters

`point` Vertex

The point to test.

Returns

bool

true if the point lies within the box; otherwise, false.

Equals(Box2)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(Box2 other)
```

Parameters

`other` Box2

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Intersects(in Box2)

Returns true if the boxes intersect.

```
public bool Intersects(in Box2 bounds)
```

Parameters

`bounds` Box2

The other box.

Returns

bool

true if the boxes intersect; otherwise, false.

IsEmpty()

Returns true if the box is empty.

```
public bool IsEmpty()
```

Returns

bool

true if the box is empty; otherwise, false.

MidPoint()

Returns the midpoint of the box.

```
public Vertex MidPoint()
```

Returns

Vertex

The midpoint.

Operators

operator ==(in Box2, in Box2)

Compares two Box2 instances for equality.

```
public static bool operator ==(in Box2 left, in Box2 right)
```

Parameters

`left` Box2

The left Box2 object.

`right` Box2

The right Box2 object.

Returns

bool

`true` if both boxes are equal; otherwise, `false`.

operator !=(in Box2, in Box2)

Determines whether two Box2 instances are not equal.

```
public static bool operator !=(in Box2 left, in Box2 right)
```

Parameters

`left` Box2

The left Box2 object.

`right` Box2

The right Box2 object.

Returns

bool

`true` if the boxes are not equal; otherwise, `false`.

Class Contour

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Represents a single polygon ring (outer contour or hole).

```
public sealed class Contour : IReadOnlyCollection<Vertex>, IEnumerable<Vertex>, IEnumerable
```

Inheritance

object ← Contour

Implements

IReadOnlyCollection<Vertex>, IEnumerable<Vertex>, IEnumerable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

A contour is treated as implicitly closed: an edge is always considered between the last vertex and the first vertex. A duplicated terminal closing vertex is optional on input but not required.

Constructors

Contour()

Initializes a new instance of the Contour class.

```
public Contour()
```

Contour(int)

Initializes a new instance of the Contour class with a vertex capacity.

```
public Contour(int capacity)
```

Parameters

`capacity` int

The initial vertex capacity.

Properties

Count

Gets the number of stored vertices.

```
public int Count { get; }
```

Property Value

int

Depth

Gets or sets the depth of the contour.

```
public int Depth { get; set; }
```

Property Value

int

HoleCount

Gets the number of holes.

```
public int HoleCount { get; }
```

Property Value

int

IsExternal

Gets a value indicating whether the contour is external (not a hole).

```
public bool IsExternal { get; }
```

Property Value

bool

this[int]

Gets the vertex at the specified index.

```
public Vertex this[int index] { get; }
```

Parameters

`index` int

The index of the vertex.

Property Value

Vertex

The Vertex at the specified index.

ParentIndex

Gets or sets the index of the parent contour in the polygon if this contour is a hole.

```
public int? ParentIndex { get; set; }
```

Property Value

int?

Methods

Add(in Vertex)

Adds a vertex to the end of the vertices collection.

```
public void Add(in Vertex vertex)
```

Parameters

`vertex` Vertex

The vertex to add.

AddHoleIndex(int)

Adds a hole index to the contour.

```
public void AddHoleIndex(int index)
```

Parameters

`index` int

The index of the hole to add.

Clear()

Clears all vertices and holes from the contour.

```
public void Clear()
```

ClearHoles()

Clears all holes from the contour.

```
public void ClearHoles()
```

DeepClone()

Creates a deep copy of this contour.

```
public Contour DeepClone()
```

Returns

Contour

A detached contour copy.

GetBoundingBox()

Gets the bounding box of the contour.

```
public Box2 GetBoundingBox()
```

Returns

Box2

The Box2.

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public IEnumerator<Vertex> GetEnumerator()
```

Returns

IEnumerator<Vertex>

An enumerator that can be used to iterate through the collection.

GetHoleIndex(int)

Gets the hole index at the specified position in the contour.

```
public int GetHoleIndex(int index)
```

Parameters

`index` int

The index of the hole.

Returns

int

The hole index.

GetLastVertex()

Gets the last vertex in the contour.

```
public Vertex GetLastVertex()
```

Returns

Vertex

The last Vertex in the contour.

IsClockwise()

Gets a value indicating whether the contour is clockwise oriented

```
public bool IsClockwise()
```

Returns

bool

true if the contour is clockwise oriented; otherwise false.

IsCounterClockwise()

Gets a value indicating whether the contour is counterclockwise oriented

```
public bool IsCounterClockwise()
```

Returns

bool

true if the contour is counterclockwise oriented; otherwise false.

RemoveVertexAt(int)

Removes the vertex at the specified index from the contour.

```
public void RemoveVertexAt(int index)
```

Parameters

`index` int

The index of the vertex to remove.

Reverse()

Reverses the orientation of the contour.

```
public void Reverse()
```

SetClockwise()

Sets the contour to clockwise orientation.

```
public void SetClockwise()
```

SetCounterClockwise()

Sets the contour to counterclockwise orientation.

```
public void SetCounterClockwise()
```

Translate(double, double)

Translates the contour by the specified x and y values.

```
public void Translate(double x, double y)
```

Parameters

`x` double

The x-coordinate offset.

y double

The y-coordinate offset.

Enum LineCap

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Specifies the shape to be used at the ends of open lines or paths when stroking.

```
public enum LineCap
```

Fields

`Butt = 0`

The stroke ends exactly at the endpoint. No extension is added beyond the path's end coordinates.

`Round = 2`

The stroke ends with a semicircular cap, extending beyond the endpoint by half the line width.

`Square = 1`

The stroke extends beyond the endpoint by half the line width, producing a square edge.

Enum LineJoin

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Specifies how the connection between two consecutive line segments (a join) is rendered when stroking paths or polygons.

```
public enum LineJoin
```

Fields

`Bevel = 3`

Joins lines by connecting the outer corners directly with a straight line, forming a flat edge at the join point.

`Miter = 0`

Joins lines by extending their outer edges until they meet at a sharp corner. If the miter limit is exceeded, the join is truncated at the limit distance.

`MiterRevert = 1`

Joins lines by extending their outer edges to form a miter. If the miter limit is exceeded, the join falls back to a bevel.

`MiterRound = 4`

Joins lines by forming a miter, but if the miter limit is exceeded, the join falls back to a round join instead of a bevel.

`Round = 2`

Joins lines by connecting them with a circular arc centered at the join point, producing a smooth, rounded corner.

Class Polygon

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Represents a complex polygon.

```
public sealed class Polygon : IReadOnlyCollection<Contour>, IEnumerable<Contour>,
IEnumerable
```

Inheritance

object ← Polygon

Implements

IReadOnlyCollection<Contour>, IEnumerable<Contour>, IEnumerable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Polygon()

Initializes a new instance of the Polygon class.

```
public Polygon()
```

Polygon(int)

Initializes a new instance of the Polygon class with a contour capacity.

```
public Polygon(int capacity)
```

Parameters

`capacity` int

The initial contour capacity.

Properties

Count

Gets the number of contours in the polygon.

```
public int Count { get; }
```

Property Value

int

this[int]

Gets the contour at the specified index.

```
public Contour this[int index] { get; }
```

Parameters

`index` int

The index of the contour.

Property Value

Contour

The Contour at the given index.

VertexCount

Gets the total number of vertices across all contours in the polygon.

```
public int VertexCount { get; }
```

Property Value

int

The total vertex count.

Methods

Add(Contour)

Adds a contour to the end of the contour collection.

```
public void Add(Contour contour)
```

Parameters

`contour` Contour

The contour to add.

Clear()

Clears all contours from the polygon.

```
public void Clear()
```

DeepClone()

Creates a deep copy of this polygon and all of its contours.

```
public Polygon DeepClone()
```

Returns

Polygon

A detached polygon copy.

GetBoundingBox()

Gets the bounding box.

```
public Box2 GetBoundingBox()
```

Returns

Box2

The Box2.

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public IEnumerator<Contour> GetEnumerator()
```

Returns

IEnumerator<Contour>

An enumerator that can be used to iterate through the collection.

GetLastContour()

Gets the last contour in the polygon.

```
public Contour GetLastContour()
```

Returns

Contour

The last Contour in the collection.

Join(Polygon)

Joins another polygon to this instance.

```
public void Join(Polygon polygon)
```

Parameters

`polygon` Polygon

The polygon to join.

ToDebugString()

Creates a string useful for debugging.

```
public string ToDebugString()
```

Returns

string

The string.

Translate(double, double)

Translates the polygon by the specified x and y values.

```
public void Translate(double x, double y)
```

Parameters

x double

The x-coordinate offset.

y double

The y-coordinate offset.

Class PolygonClipper

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Performs boolean operations on polygons.

```
public class PolygonClipper
```

Inheritance

object ← PolygonClipper

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

This implementation follows the algorithm described in "A Simple Algorithm for Boolean Operations on Polygons" by Francisco Martínez, Carlos Ogayar, Juan R. Jiménez, and Antonio J. Rueda. It supports intersection, union, difference, and symmetric difference (XOR).

It uses a sweep-line with an event queue to process segment intersections and robustly handles special cases, including overlapping edges and trivial non-overlapping inputs.

The static boolean methods are the recommended entry points. They route work through an internal thread-local pool of reusable clipper instances and automatically reset temporary state between calls.

Instance members are not thread-safe for concurrent use.

The high-level workflow has three stages:

1. Preprocessing: Handles trivial operations and prepares segments for processing.
2. Sweeping: Processes events using a priority queue, handling segment insertions and removals.
3. Connecting edges: Constructs the resulting polygon by connecting valid segments.

Constructors

PolygonClipper(Polygon, Polygon, BooleanOperation)

Initializes a new instance of the PolygonClipper class.

```
public PolygonClipper(Polygon subject, Polygon clip, BooleanOperation operation)
```

Parameters

`subject` Polygon

The polygon used as the left-hand operand.

`clip` Polygon

The polygon used as the right-hand operand.

`operation` BooleanOperation

The boolean operation to execute.

Remarks

This constructor is intended for advanced/manual execution flows. For typical usage, prefer the static methods to take advantage of internal pooling and automatic lifecycle management.

Methods

Difference(Polygon, Polygon)

Computes the difference of two polygons (`subject` minus `clip`).

```
public static Polygon Difference(Polygon subject, Polygon clip)
```

Parameters

`subject` Polygon

The polygon used as the left-hand operand.

`clip` Polygon

The polygon used as the right-hand operand.

Returns

Polygon

A polygon containing regions from `subject` not covered by `clip`.

Remarks

Preferred entry point. Uses internal thread-local reusable instances.

Intersection(Polygon, Polygon)

Computes the intersection of two polygons.

```
public static Polygon Intersection(Polygon subject, Polygon clip)
```

Parameters

`subject` Polygon

The polygon used as the left-hand operand.

`clip` Polygon

The polygon used as the right-hand operand.

Returns

Polygon

A polygon containing regions common to both inputs.

Remarks

Preferred entry point. Uses internal thread-local reusable instances.

Normalize(Polygon)

Normalizes a polygon by resolving self-intersections and overlaps.

```
public static Polygon Normalize(Polygon polygon)
```

Parameters

`polygon` Polygon

The polygon to process.

Returns

Polygon

A new normalized polygon. Output contours are implicitly closed (no duplicated terminal closing vertex is appended).

Remarks

Preferred entry point. Uses internal thread-local reusable builders.

Run()

Executes the configured boolean operation for the current subject and clipping polygons.

```
public Polygon Run()
```

Returns

Polygon

The operation result.

Remarks

Instance execution is not thread-safe for concurrent use.

Union(Polygon, Polygon)

Computes the union of two polygons.

```
public static Polygon Union(Polygon subject, Polygon clip)
```

Parameters

`subject` Polygon

The polygon used as the left-hand operand.

`clip` Polygon

The polygon used as the right-hand operand.

Returns

Polygon

A polygon containing regions from either input.

Remarks

Preferred entry point. Uses internal thread-local reusable instances.

Xor(Polygon, Polygon)

Computes the symmetric difference (XOR) of two polygons.

```
public static Polygon Xor(Polygon subject, Polygon clip)
```

Parameters

`subject` Polygon

The polygon used as the left-hand operand.

`clip` Polygon

The polygon used as the right-hand operand.

Returns

Polygon

A polygon containing regions that belong to exactly one input.

Remarks

Preferred entry point. Uses internal thread-local reusable instances.

Class PolygonStroker

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Generates polygonal stroke geometry for contours with configurable joins and caps.

```
public sealed class PolygonStroker
```

Inheritance

object ← PolygonStroker

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

This type performs two phases:

1. Expand each source contour into one or two stroke-side outlines with joins/caps.
2. Optionally resolve generated overlaps/self-intersections using Normalize(Polygon) with positive fill semantics.

The emitted contours are implicitly closed (first vertex is not duplicated at the end).

The static Stroke(Polygon, double, StrokeOptions?) method is the recommended entry point. It routes calls through an internal thread-local pool of reusable stroker instances and automatically resets temporary state between calls.

Instance members are not thread-safe for concurrent use.

Constructors

PolygonStroker(StrokeOptions)

Initializes a new instance of the PolygonStroker class with the specified stroke options.

```
public PolygonStroker(StrokeOptions options)
```

Parameters

`options` StrokeOptions

The stroke options.

Remarks

This constructor is intended for advanced/manual usage. For typical call patterns, prefer the static Stroke(Polygon, double, StrokeOptions?) method to use internal pooling automatically.

Exceptions

ArgumentNullException

Thrown when `options` is null.

Properties

ArcDetailScale

Gets the tessellation detail scale used for round joins and round caps. Higher values produce more vertices and smoother curves.

```
public double ArcDetailScale { get; }
```

Property Value

double

LineCap

Gets the line cap style used for open path ends.

```
public LineCap LineCap { get; }
```

Property Value

LineCap

LineJoin

Gets the outer line join style used for stroking corners.

```
public LineJoin LineJoin { get; }
```

Property Value

LineJoin

MiterLimit

Gets the miter limit used to clamp outer miter joins.

```
public double MiterLimit { get; }
```

Property Value

double

NormalizeOutput

Gets a value indicating whether generated contours should be normalized by resolving self-intersections and overlaps.

```
public bool NormalizeOutput { get; }
```

Property Value

bool

Width

Gets or sets the stroke width.

```
public double Width { get; set; }
```

Property Value

double

Remarks

Positive values produce conventional outward stroking. Negative values are supported and flip the side orientation while preserving magnitude.

Methods

Stroke(Polygon)

Strokes `polygon` using this instance's configured options and width.

```
public Polygon Stroke(Polygon polygon)
```

Parameters

`polygon` Polygon

Input polygon to stroke.

Returns

Polygon

The stroked polygon contours.

Remarks

Instance execution is not thread-safe for concurrent use.

Exceptions

ArgumentNullException

Thrown when `polygon` is null.

Stroke(Polygon, double)

Strokes `polygon` after setting the current stroke width.

```
public Polygon Stroke(Polygon polygon, double width)
```

Parameters

`polygon` Polygon

Input polygon to stroke.

`width` double

Stroke width.

Returns

Polygon

The stroked polygon contours.

Remarks

Instance execution is not thread-safe for concurrent use.

Stroke(Polygon, double, StrokeOptions?)

Strokes `polygon` with `width` using optional `options`.

```
public static Polygon Stroke(Polygon polygon, double width, StrokeOptions? options = null)
```

Parameters

`polygon` Polygon

Input polygon to stroke.

`width` double

Stroke width.

`options` StrokeOptions

Stroke options controlling joins, caps and approximation behavior. When null, default Stroke Options are used.

Returns

Polygon

The stroked polygon contours.

Remarks

Preferred entry point. Uses internal thread-local reusable instances.

Exceptions

ArgumentNullException

Thrown when `polygon` is null.

Enum ResultTransition

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Represents the result transition for a sweep event.

```
public enum ResultTransition
```

Fields

`Contributing = 1`

The event contributes to the result.

`Neutral = 0`

The event transitions within the result.

`NonContributing = -1`

The event does not contribute to the result.

Class StrokeOptions

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Provides configuration options for geometric stroke generation.

```
public sealed class StrokeOptions : IEquatable<StrokeOptions?>
```

Inheritance

object ← StrokeOptions

Implements

IEquatable<StrokeOptions>

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

ArcDetailScale

Gets or sets the tessellation detail scale for round joins and round caps. Higher values produce more vertices (smoother curves, more work). Lower values produce fewer vertices.

```
public double ArcDetailScale { get; set; }
```

Property Value

double

LineCap

Gets or sets the line cap style used for open path ends.

```
public LineCap LineCap { get; set; }
```

Property Value

LineCap

LineJoin

Gets or sets the outer line join style used for stroking corners.

```
public LineJoin LineJoin { get; set; }
```

Property Value

LineJoin

MiterLimit

Gets or sets the miter limit used to clamp outer miter joins.

```
public double MiterLimit { get; set; }
```

Property Value

double

NormalizeOutput

Gets or sets a value indicating whether stroked contours should be normalized by resolving self-intersections and overlaps before returning.

```
public bool NormalizeOutput { get; set; }
```

Property Value

bool

Remarks

Defaults to false for maximum throughput. When disabled, callers should rasterize with a non-zero winding fill rule.

Methods

Equals(StrokeOptions?)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(StrokeOptions? other)
```

Parameters

`other` StrokeOptions

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current object.

Returns

bool

true if the specified object is equal to the current object; otherwise, false.

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

int

A hash code for the current object.

Struct Vertex

Namespace: SixLabors.PolygonClipper

Assembly: SixLabors.PolygonClipper.dll

Represents a two-dimensional vertex with X and Y coordinates.

```
public readonly struct Vertex : IEquatable<Vertex>
```

Implements
IEquatable<Vertex>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Vertex(double)

Initializes a new instance of the Vertex struct.

```
public Vertex(double xy)
```

Parameters

xy double

The X and Y coordinates of the vertex.

Vertex(double, double)

Initializes a new instance of the Vertex struct.

```
public Vertex(double x, double y)
```

Parameters

x double

The X-coordinate of the vertex.

y double

The Y-coordinate of the vertex.

Fields

X

Gets the X-coordinate of the vertex.

```
public readonly double X
```

Field Value

double

Y

Gets the Y-coordinate of the vertex.

```
public readonly double Y
```

Field Value

double

Methods

Abs(in Vertex)

Computes the absolute value of each element in a specified vertex.

```
public static Vertex Abs(in Vertex value)
```

Parameters

`value` Vertex

The vertex that will have its absolute value computed.

Returns

Vertex

A vertex with the absolute value of each of the elements in `value`.

Cross(in Vertex, in Vertex)

Returns the cross product of two vertices.

```
public static double Cross(in Vertex a, in Vertex b)
```

Parameters

`a` Vertex

The first vertex.

`b` Vertex

The second vertex.

Returns

double

The double cross product.

Distance(in Vertex, in Vertex)

Computes the Euclidean distance between the two given vertices.

```
public static double Distance(in Vertex a, in Vertex b)
```

Parameters

a Vertex

The first vertex.

b Vertex

The second vertex.

Returns

double

The distance.

DistanceSquared(in Vertex, in Vertex)

Returns the Euclidean distance squared between two specified vertices.

```
public static double DistanceSquared(in Vertex a, in Vertex b)
```

Parameters

a Vertex

The first vertex.

b Vertex

The second vertex.

Returns

double

The distance squared.

Dot(in Vertex, in Vertex)

Returns the dot product of two vertices.

```
public static double Dot(in Vertex a, in Vertex b)
```

Parameters

`a` Vertex

The first vertex.

`b` Vertex

The second vertex.

Returns

double

The double dot product.

Equals(Vertex)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(Vertex other)
```

Parameters

`other` Vertex

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Length()

Returns the length of the vertex.

```
public double Length()
```

Returns

double

The vertex's length.

LengthSquared()

Returns the length of the vertex squared.

```
public double LengthSquared()
```

Returns

double

The vertex's length squared.

Remarks

This operation offers better performance than a call to the Length() method.

Max(in Vertex, in Vertex)

Returns a vertex whose elements are the maximum of each of the pairs of elements in two specified vertices.

```
public static Vertex Max(in Vertex a, in Vertex b)
```

Parameters

a Vertex

The first vertex.

b Vertex

The second vertex.

Returns

Vertex

The maximized Vertex.

Min(in Vertex, in Vertex)

Returns a vertex whose elements are the minimum of each of the pairs of elements in two specified vertices.

```
public static Vertex Min(in Vertex a, in Vertex b)
```

Parameters

a Vertex

The first vertex.

b Vertex

The second vertex.

Returns

Vertex

The minimized Vertex.

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

Operators

operator +(in Vertex, in Vertex)

Adds two vectors together.

```
public static Vertex operator +(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vector to add.

`right` Vertex

The second vector to add.

Returns

Vertex

The summed vector.

Remarks

The operator +(in Vertex, in Vertex) method defines the addition operation for Vector2 objects.

operator /(in Vertex, in Vertex)

Divides the first vertex by the second.

```
public static Vertex operator /(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vertex.

`right` Vertex

The second vertex.

Returns

Vertex

The vertex that results from dividing `left` by `right`.

operator /(in Vertex, double)

Divides the specified vertex by a specified scalar value.

```
public static Vertex operator /(in Vertex left, double right)
```

Parameters

`left` Vertex

The first vertex.

`right` double

The scalar value.

Returns

Vertex

The result of the division.

operator /(double, in Vertex)

Divides the specified vertex by the specified scalar value.

```
public static Vertex operator /(double left, in Vertex right)
```

Parameters

`left` double

The first vertex.

`right` Vertex

The scalar value.

Returns

Vertex

The result of the division.

operator ==(in Vertex, in Vertex)

Determines whether two vertices are equal.

```
public static bool operator ==(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vertex.

`right` Vertex

The second vertex.

Returns

bool

true if the vertices are equal; otherwise, false.

operator !=(in Vertex, in Vertex)

Determines whether two vertices are not equal.

```
public static bool operator !=(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vertex.

`right` Vertex

The second vertex.

Returns

bool

true if the vertices are not equal; otherwise, false.

operator *(in Vertex, in Vertex)

Returns a new vector whose values are the product of each pair of elements in two specified vertices.

```
public static Vertex operator *(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vertex.

`right` Vertex

The second vertex.

Returns

Vertex

The element-wise product vertex.

operator *(in Vertex, double)

Multiplies the specified vertex by the specified scalar value.

```
public static Vertex operator *(in Vertex left, double right)
```

Parameters

`left` Vertex

The first vertex.

`right` double

The scalar value.

Returns

Vertex

The scaled vertex.

operator *(double, in Vertex)

Multiplies the specified vertex by the specified scalar value.

```
public static Vertex operator *(double left, in Vertex right)
```

Parameters

`left` double

The first vertex.

`right` Vertex

The scalar value.

Returns

Vertex

The scaled vertex.

operator -(in Vertex, in Vertex)

Subtracts the second vector from the first.

```
public static Vertex operator -(in Vertex left, in Vertex right)
```

Parameters

`left` Vertex

The first vector.

`right` Vertex

The second vector.

Returns

Vertex

The vector that results from subtracting `right` from `left`.

Remarks

The operator -(in Vertex, in Vertex) method defines the subtraction operation for Vector2 objects.